

Security & Compliance — org-analyser

What this is: the security and compliance posture of the pipeline as it stands today — what data it captures, where that data goes, what controls constrain it, and what is still on you rather than on the code.

Date: 2026-07-13 · **Method:** manual source review of every pipeline entry point and data flow, plus two executable tests that assert the key guarantees. No dependency CVE scan and no third-party attestation — see §8.

Posture in one paragraph. The pipeline is safe to run **on codebases you own** (§5.1). Its two structural risks — untrusted repository code executing with your credentials, and repository content reaching third-party LLMs — are both closed by controls that hold at a choke point rather than by convention, and both are covered by tests, including an end-to-end run against a deliberately hostile repository (§4, §8). Personal data is minimised before it reaches any deliverable, and run bundles now expire. One item remains open and **it is not a code change**: confirm your Gemini API key's tier, because a free-tier key means your source code is used for Google's model training (§3, §7).

1. What the system does, and its trust model

`run_org_pipeline.py` orchestrates six phases against a GitHub org, GitLab group, or explicit repo list. For each repo it clones the source, runs static profilers, calls LLMs to classify and score, and writes a zipped report bundle to `outputs/org-pipeline-runs/`.

Three properties follow from that shape, and they define the whole threat model:

1. **The inputs are untrusted.** Not because you don't trust your own org — but because a repository contains its dependencies' code, its external contributors' branches, and free text anyone could write into a PR comment. "We own this repo" is not the same as "everything in this repo is code we wrote."
2. **The credentials are broad.** An org-wide read token plus paid LLM keys, held by a process that runs other people's build scripts.
3. **The outputs are personal data.** Who wrote what, when, and how good it was.

Controls in §4 map one-to-one onto these three.

2. What data is captured

2.1 Repository content

Data	Captured by	Stored where	Retained
Full repository clone (all files, full git history)	<code>fresh_clone</code> (<code>run_org_pipeline.py</code>)	<code>outputs/.../clones/</code> or <code>~/org-pipeline-clones/</code>	Deleted after every run (<code>remove_clones</code>)
Code samples, file paths, LOC, duplication, complexity	<code>codebase_profiler</code> , <code>repo_analyzer</code>	Run bundle (XLSX/CSV/JSON)	Retention window (§4.5)
Git diffs / patches	<code>repo-eval-kit</code>	Sent to LLM; scores persisted	Scores only
Secrets detected in scanned code	<code>scan_secrets_and_pii</code> (<code>repo_analyzer.py</code>)	Run bundle, masked as <code>[REDACTED]</code>	Masked only — the value is never written

2.2 Personal data

The pipeline reads more personal data than it keeps. The distinction matters, so both columns are stated:

Data	Read from	Written to the deliverable?
Committer names	git history (<code>%an</code>)	Yes — <code>top_authors[].name</code>
Committer email addresses	git history (<code>%ae</code>)	No . Used transiently for bot detection, then replaced at emission with a stable pseudonym: <code>author_id: anon:ced44e91...</code> (§4.6)
PR/MR author logins	GitHub/ GitLab API	Yes — pseudonymous, but re-identifying
Human code-review comment bodies	GitHub/ GitLab API	Sent to the LLM for classification (§3); not persisted verbatim
Per-author contribution statistics	git history	Yes — commit counts, <code>is_bot</code> , derived scores
Email addresses appearing <i>in code</i>	file scan	Count only , never the addresses

This system profiles individual developers. Author-attributed activity and per-person statistics are personal data under GDPR Art. 4(1), and pseudonymisation reduces that exposure without eliminating it — a name plus a commit count is still about a person. The compliance consequences are in §5.

2.3 Credentials handled

`github-data-token` , `gitlab_token` , `openai_key` — read from a gitignored `tokens` file or `.env` , both mode `0600` . No live credential appears in any committed file or in the `outputs/` tree (verified by scan). Runtime handling is covered by the controls in §4.1–§4.3.

3. Where data goes — third-party disclosure

Destination	What is sent	Redacted first?
<code>api.github.com / gitlab.com / api.bitbucket.org</code>	Authenticated read requests	n/a — these are the source systems
<code>api.openai.com</code>	Code samples, git diffs, PR titles and bodies, human review comments	Yes — at the client boundary (§4.2)
<code>generativelanguage.googleapis.com</code> (Gemini)	Source and test diffs	Yes (§4.2)

Both providers are **processors** in GDPR terms and belong in a Record of Processing Activities. Both are US-based, so EU-resident developer data needs a transfer basis (SCCs / Data Privacy Framework).

Two provider-level facts that redaction cannot address, because they are about the contract, not the payload:

- **OpenAI API** does not train on API data by default, but retains it up to 30 days for abuse monitoring unless you hold a Zero Data Retention agreement.
- **Google Gemini** — on the **free/unpaid tier**, Google *does* use submitted content to improve its products. If the key configured in `quality_evaluator.py` is free-tier, your source code is going into Google's training data. **This is the single highest-consequence open question in the system** (§7).

4. Controls

Each control is stated as the guarantee it provides, where it is enforced, and how you can check it still holds. Controls that depend on every caller remembering to do the right thing are not controls, so each of these sits at a choke point.

4.1 Repository code cannot read your credentials

The F2P/P2P analysis runs the target repo's own build and test commands — `make`, `pytest`, `npm`, `gradle`, `dotnet` — inside the clone. That is arbitrary code execution by design, and it is the price of the feature.

Guarantee: that code runs with a **default-deny environment**. `build_child_env()` (`test_runners/base.py`) constructs the child env from an allowlist — `PATH`, `HOME`, locale, temp dirs, Windows essentials, toolchain roots (`JAVA_HOME`, `GOPATH`, `CARGO_HOME`, ...). Everything else is dropped, including every credential the pipeline holds.

Why it holds: `_run_command` is the single choke point every test runner routes through. There is no second path — `analyzer.py`'s other subprocess calls are all plain `git`, and the one caller passing a custom env passes an overrides dict, not a fresh `os.environ.copy()`.

Check: `python test_env_scrub.py` — seeds five fake credentials, spawns a child that dumps its own environment, asserts none survive.

Residual risk, accepted. Repo test code still executes on the host as your user. It can no longer read your tokens from the environment, but it can still reach the network and read files your account can read (`~/.ssh`, `~/.aws/credentials`). This is the same trust boundary as any CI runner executing repo tests. If you ever analyse repos taking untrusted external contributions, containerise this phase (`--network=none`) rather than relying on the env scrub alone.

Knob: the allowlist is deliberately tight, so a build needing an extra variable fails loudly rather than leaking silently. Widen it per-run without editing code: `F2P_ENV_PASSTHROUGH="ARTIFACTORY_URL,F00_HOME"`.

4.2 Secrets cannot reach an LLM provider

Guarantee: every OpenAI client in the repo is constructed by `llm_safety.safe_openai()`, which returns a guarded client that redacts `messages` (and Responses-API `input`) **on the way out**. Redaction is a property of the client, not a step a call site has to remember.

Why it holds: redaction used to be opt-in per call site, and was wired into only four of seven — which is precisely how raw git diffs, PR bodies, and human review comments came to be shipped unredacted. Moving it into the client removes the failure mode rather than patching its instances.

Check: `grep -rn '= OpenAI(' .` returns nothing outside the wrapper itself. `python test_llm_redaction.py` drives a fake client and asserts no key material reaches it.

Quality is unaffected. Only secret *values* are replaced. Prose, code structure, diff markers and identifiers pass through untouched — asserted explicitly in `test_wrapper_redacts_in_flight`.

Honest limit: the redactor is a **pattern allowlist** (vendor token formats, PEM blocks, AWS secret keys, DB connection strings). It will never catch every bespoke internal token format. It is defence-in-depth, not a guarantee — do not describe it as one in a customer-facing claim.

4.3 Tokens do not leak through argv, disk, or logs

Three paths, all closed:

- **argv** — the token reaches the eval-kit as `REPO_EVAL_TOKEN` in the child's environment, never as `--token`. Argv is world-readable via `ps` and `/proc/<pid>/cmdline`.
- **Clone URL** — git auth uses the env-config channel (`GIT_CONFIG_COUNT / KEY_0 / VALUE_0` → `http.extraHeader`). A tokenised URL would be written verbatim into `<clone>/.git/config`; `git -c http.extraHeader=...` would have put it back on argv. The env channel leaks through neither. (Verified: `git config --get http.extraHeader` reads the value back.)
- **Error output** — clone stderr passes through `scrub_secrets()` before it reaches an exception or a log.

4.4 LLM output is never executed

No `eval`, `exec`, `pickle.load`, or unsafe `yaml.load` anywhere in the tree (verified). This bounds the blast radius of §6 (prompt injection) to score integrity — it cannot reach code execution.

4.5 Run bundles expire

`--retention-days` (default **90**) sweeps run folders older than the window before each run; `0` disables it. Bundles carry contributor names, per-author statistics and scores, so "keep forever" was a storage-limitation problem with no deletion path. Now there is one.

4.6 Committer emails do not reach the deliverable

Emails are read from git history, used for bot detection, and then replaced at emission with a stable pseudonym (`author_id: anon:...`, `git_stats.py`).

Zero quality cost, and this is worth being precise about: bot detection already runs against the *real* address and stores its verdict as `is_bot`, and nothing downstream ever read the address itself. The hash is stable, so authors can still be deduplicated and joined across repos and runs. Nothing that consumed this data lost anything.

5. Compliance posture

5.1 Deployment model

This tool is run by the owner of the codebases being analysed, on their own repositories, with their authorisation. That is the supported model, and it settles what would otherwise be the dominant questions:

- **Authorisation to execute repo code** — granted; the operator owns the code being run.
- **Authorisation to disclose source to OpenAI/Google** — granted by the operator, for their own source.
- **Purpose** — internal engineering assessment of the operator's own codebases.

Running this against a repository you do **not** own or control is out of scope and unsupported: neither authorisation holds, and §4.1's residual risk stops being acceptable.

5.2 Remaining obligations

Under the owner-run model, the surviving compliance obligations are operational, and all but one are already met in code:

Requirement	Status
Data minimisation (Art. 5(1)(c))	Met. Emails pseudonymised at emission; in-code emails counted, not stored (§4.6, §2.2).
Storage limitation (Art. 5(1)(e))	Met. 90-day default retention sweep; clones deleted every run (§4.5).
Security of processing (Art. 32)	Met for the identified findings — §4.1 through §4.3, all test-covered. Accepted: prompt injection (§6).
Processors / transfers (Art. 28, Ch. V)	Confirm once. Repository content reaches OpenAI and Google. Standard API DPAs almost certainly cover this — but confirm the Gemini tier (§3): free-tier submission puts your source into Google's training.

6. Known limitations, accepted

- **Prompt injection is unmitigated.** Untrusted repository content flows into LLM prompts whose outputs drive scoring. A repository can embed instructions to inflate its own score or suppress a finding. Bounded by §4.4 — it cannot reach code execution — and under the owner-run model the incentive to attack your own score is low. Treat scores from unreviewed repos as advisory.
- **Redaction is pattern-based** (§4.2) and will not catch every bespoke secret format.
- **Test code runs on the host**, not in a container (§4.1).
- `shell=True` at [bulk_repo_evaluator_parallel.py:320](#) — fed internally-constructed commands today, but a shell-injection primitive waiting for a future caller.
- **No dependency vulnerability scanning.** `requirements.txt` pins only lower bounds. `pip-audit` in CI would close this.

7. Open items — operator decisions

1. **Confirm the Gemini API key's tier.** If it is free-tier, Google uses your source code for model training (§3). Not a code change, and the highest-consequence unknown in the system.

2. **Historical run bundles predate the email fix.** Bundles already in `outputs/org-pipeline-runs/` still contain real committer addresses — §4.6 applies to new runs only. Purge them or re-run.
-

8. Verification

```
$ cd repo-eval-kit

$ python test_env_scrub.py
OK: repo commands run credential-free

$ python test_llm_redaction.py
ok test_common_tokens_redacted
ok test_private_key_fully_redacted
ok test_private_key_in_diff
ok test_stats_never_overclaim
ok test_wrapper_redacts_in_flight
OK: no secret reaches an LLM provider

$ python test_hostile_repo.py
runner: pytest
hostile conftest.py ran and captured 11 env vars
OK: hostile repo executed, stole nothing

$ python -m pip_audit
No known vulnerabilities found
```

- `test_env_scrub.py` seeds five fake credentials and spawns a child that dumps its own environment, asserting none survive — a unit test of the allowlist boundary.
- `test_hostile_repo.py` goes further: it builds a repository whose `conftest.py` exfiltrates its environment to disk (exactly what a supply-chain `postinstall` or a hostile contributor's branch would do), drives the **real** `PytestRunner` against it, confirms the payload actually executed, and asserts the credentials never reached it. This is the dynamic hostile-repo test — the attack path exercised end to end.
- `test_llm_redaction.py` drives a fake OpenAI client and asserts no key material reaches the provider, *and* that surrounding prose and diff structure survive, so classification quality is unchanged.
- `pip-audit` reports no known CVEs in the pinned dependency tree.

Run the three tests after any change to the LLM call sites, the redactor, or the test runners; re-run `pip-audit` when dependencies change.

Appendix — hardening history

All of the following were live defects found in the 2026-07-13 review of commit `cd51f4a`, and were fixed the same day. Recorded because two of them were **silently wrong rather than merely absent**, which is the failure mode most worth remembering.

Defect	Why it mattered
Repo test commands inherited <code>os.environ.copy()</code>	A hostile <code>conftest.py</code> or <code>npm postinstall</code> could read the org GitHub token and OpenAI key straight out of its environment. On by default.
Redaction opt-in per call site, wired into 4 of 7	Raw git diffs, PR bodies, and human review comments went to OpenAI unredacted.
PEM regex matched only the <code>-----BEGIN-----</code> line	Key material was transmitted while the stats reported ('Private Key', 1) . Worse than no redaction — it produced false assurance in the logs. <code>redact_diff</code> compounded it by scanning line-by-line, which can never match a multi-line PEM at all.
Counts from <code>findall</code> on the original text	Stats could claim a redaction that never happened. Now <code>re.subn</code> on the running text.
Token passed as <code>--token</code> on argv	World-readable via <code>ps</code> .
Token embedded in clone URL	Persisted into <code>.git/config</code> ; echoed back by git's auth-failure stderr, which was raised raw.
Gemini key in the URL query string	Lands in proxy logs, gateway logs, shell history.
<code>GIT_ASKPASS_OVERRIDE</code> / <code>GIT_HTTP_EXTRAHEADER</code>	Not real git variables. Git never read them, so private clones silently fell back to ambient host credentials. A live functional bug, not only a security one.
<code>err_msg.replace("", "[REDACTED])"</code>	With no token set, spliced the marker between every character of the message.
Committer emails written to <code>top_authors[].email</code>	Real developer addresses in the deliverable, for no analytical gain.
<code>tokens</code> / <code>.env</code> at mode <code>0644</code>	World-readable secrets.